

Implementación y análisis de complejidad de listas, pilas y colas en Java

Hayran Andrés López González

Objetivo del taller

Implementar las estructuras de datos List (listas enlazadas), Stack (pila) y Queue (cola) en Java, abarcando tanto arreglos dinámicos como listas enlazadas, realizando un análisis de complejidad de los métodos asociados.

Explicación de la implementación

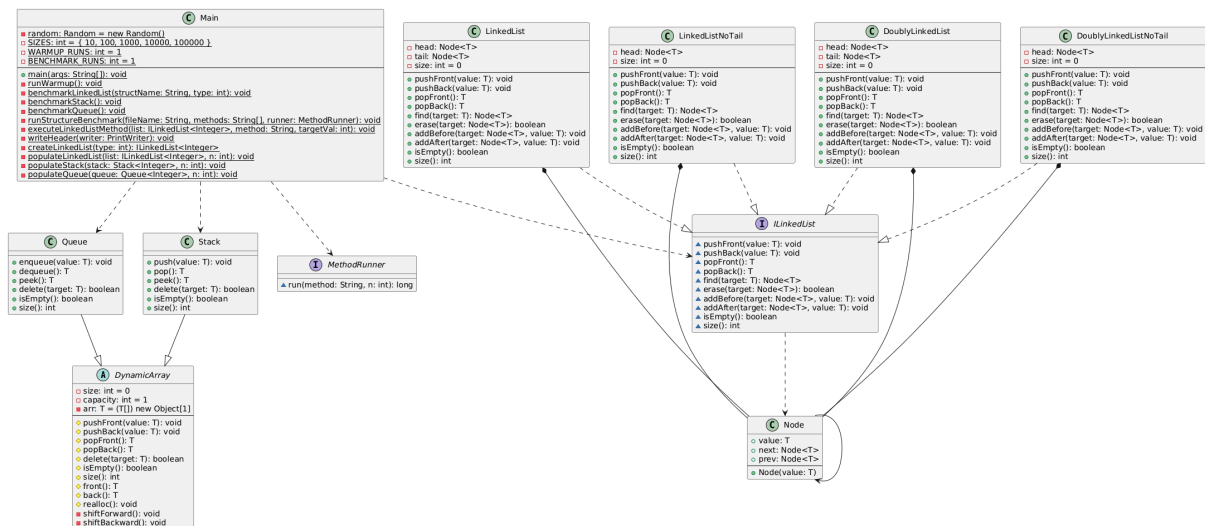
Para la implementación de este ejercicio se ha seguido la siguiente estructura:

- Una interfaz `ILinkedList` con los métodos comunes para todas las listas enlazadas (insertar, eliminar, buscar, etc.).
- Una clase abstracta `DynamicArray` con los métodos de un arreglo dinámico ya implementados. Es abstracta porque no la usaremos directamente.

De aquí parten las clases que si se usan:

- Las 4 variantes de listas enlazadas, todas implementando la interfaz `ILinkedList`:
 - `LinkedList`
 - `LinkedListNoTail`
 - `DoublyLinkedList`
 - `DoublyLinkedListNoTail`
- `Stack` y `Queue` para las pilas y colas respectivamente

El diagrama UML de este proyecto es el siguiente:



Análisis de complejidad

Listas enlazadas

Simple

- Buscar siempre toma $O(n)$, y es el cuello de botella de los siguientes métodos
- Insertar y eliminar al frente es inmediato porque tenemos un puntero a la cabeza de la lista y el siguiente nodo. Complejidad $O(1)$
- Insertar y eliminar al final requiere buscar el último nodo y el penúltimo (al eliminar), por lo que la complejidad es $O(n)$
- Añadir antes de un nodo requiere buscar el anterior ($O(n)$) pero añadir después se puede hacer de inmediato porque tenemos acceso al siguiente nodo ($O(1)$)
- Eliminar un nodo requiere buscar el anterior, entonces su complejidad es $O(n)$
- Verificar que está vacío es tan sencillo como ver si hay cabeza. Complejidad $O(1)$

Ten en cuenta que los métodos `addBefore` y `addAfter` ofrecen un puntero directo al nodo, entonces por ello pueden ser más rápidos.

Simple (con cola)

El nuevo puntero con cola hace que añadir elementos al final sea más sencillo porque tenemos la cola directamente. Pero el resto de operaciones siguen igual en cuanto requieren búsquedas (especialmente para buscar el nodo anterior a uno dado).

Dobles

Las listas enlazadas dobles introducen un puntero anterior que nos permite trabajar hacia atrás también. Gracias a eso, ahora borrar nodos y añadir antes de uno puede hacerse en tiempo constante porque podemos ver hacia atrás y cablear directamente.

Pero como no tienen puntero a cola, aún debemos buscarla, degradando las operaciones al final de la lista.

Dobles (con cola)

Ahora con el puntero a cola, podemos ir directo al final de la lista y operar sobre ella, haciendo que la adición/eliminación de elementos se torne de tiempo constante.

Con estas consideraciones, la tabla de complejidades queda como se ve a continuación. En azul las operaciones con rendimiento mejorado y en rojo las que quedan peor a comparación.

Operación	Simple	Simple (con cola)	Doble	Doble (con cola)
<code>pushFront</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<code>pushBack</code>	$O(n)$	$O(1)$	$O(n)$	$O(1)$
<code>popFront</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<code>popBack</code>	$O(n)$	$O(n)$	$O(n)$	$O(1)$
<code>find</code>	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<code>erase</code>	$O(n)$	$O(n)$	$O(1)$	$O(1)$
<code>addBefore</code>	$O(n)$	$O(n)$	$O(1)$	$O(1)$
<code>addAfter</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<code>isEmpty</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$

Comparando las tablas obtenemos que la mejor estructura en términos de rendimiento general es la **lista enlazada doble con puntero a cola**, al permitir un avance en dos direcciones y acceso a los dos extremos.

Arreglos dinámicos, pilas y colas

Los arreglos dinámicos son arreglos cuyo tamaño puede crecer de manera indefinida, permitiendo acceder a elementos de manera aleatoria (por índice) y reasignarse automáticamente cuando se llena.

Eliminar o añadir un elemento de una posición diferente a la última requiere trasladar el resto de elementos hacia adelante o hacia atrás, lo cual hace que su complejidad sea de $O(1)$. El resto de operaciones pueden hacerse de manera inmediata y por ende con $O(1)$.

La complejidad de sus operaciones es la siguiente:

Operación	Pila	Cola	Complejidad
pushFront	-	-	$O(n)$
pushBack	push	enqueue	$O(1)$
popFront	-	dequeue	$O(n)$
popBack	pop	-	$O(1)$
delete	delete	delete	$O(n)$
isEmpty	isEmpty	isEmpty	$O(1)$
size	size	size	$O(1)$
front	-	front	$O(1)$
back	peek	-	$O(1)$

Ten en cuenta que se están usando arreglos dinámicos llenados de inicio a fin. Con arreglos circulares se puede lograr más rendimiento, y con arreglos circulares dinámicos (no implementados en este caso) también una capacidad máxima arbitraria.

Gráficas y resultados empíricos

Para analizar el comportamiento real de estos métodos utilizaremos medidas basadas en `java.time.Instant` y `java.time.Duration`. El esquema de medición es el siguiente:

- Mediremos el rendimiento con 10, 100, 1000, ..., hasta 10^{10} operaciones
- Esta medición es por cada método de cada clase implementada
- Haremos la tabla de tiempos y gráficas para los mismos. Una tabla por cada estructura de datos
- Las gráficas serán en escala logarítmica para evidenciar todas las escalas adecuadamente. Una gráfica por cada tabla
- Se usarán **nanosegundos** para las mediciones

Funcionamiento de la Suite:

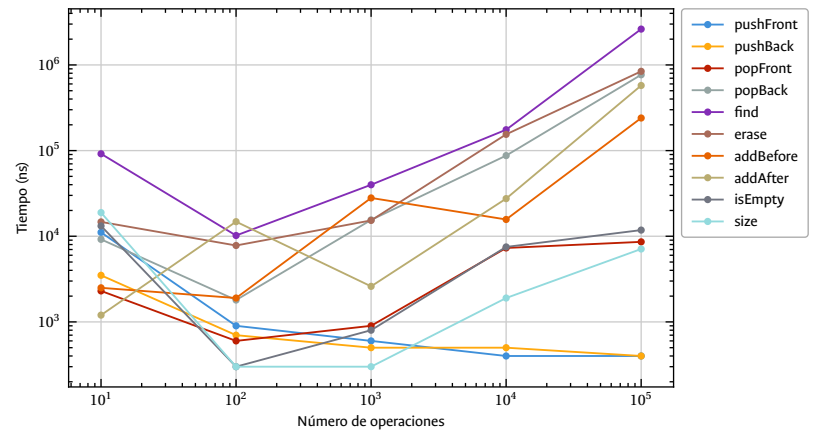
1. **JVM Warmup:** Ejecuta pasadas iniciales descartadas para permitir que la JVM realice optimizaciones JIT.
2. **Escala Exponencial (N):** Evalúa cada método con tamaños $N \in \{10, 100, 1000, 10000, 100000\}$.
3. **Población con Datos Aleatorios:** Se llenan las estructuras previamente usando valores aleatorios para métodos de eliminación o consulta.
4. **Archivos CSV Matriciales Separados:** Se genera un archivo .csv independiente por cada estructura de datos.

El código para registrar estos tiempos no interfiere con los algoritmos, y se ubica en la clase `Main`. Las gráficas se crean con **lilaq**, una librería de **Typst**, el mismo sistema con el que se ha compilado este PDF. los datos para lograrlo se registrarán en un CSV.

Listas enlazadas

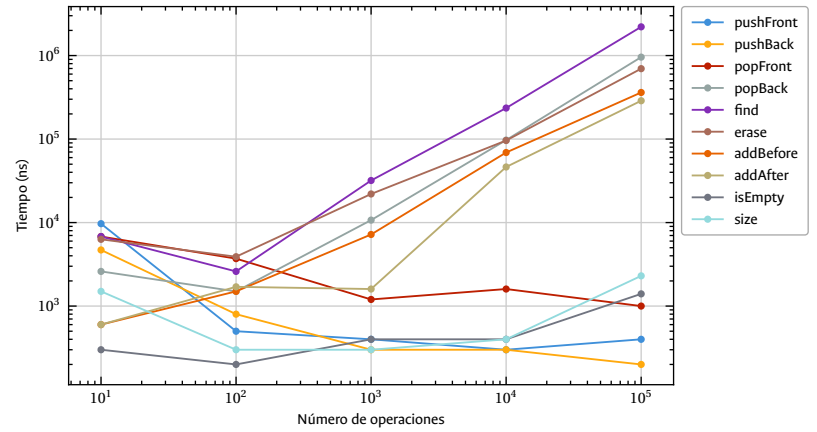
1. Simples

Method	10	100	1000	10000	100000
pushFront	11100	900	600	400	400
pushBack	3500	700	500	500	400
popFront	2300	600	900	7300	8600
popBack	9200	1800	15500	87400	767400
find	91800	10200	39900	175200	2621200
erase	14700	7800	15300	154700	842100
addBefore	2500	1900	28000	15700	240100
addAfter	1200	14800	2600	27500	575500
isEmpty	13200	300	800	7500	11800
size	18900	300	300	1900	7100



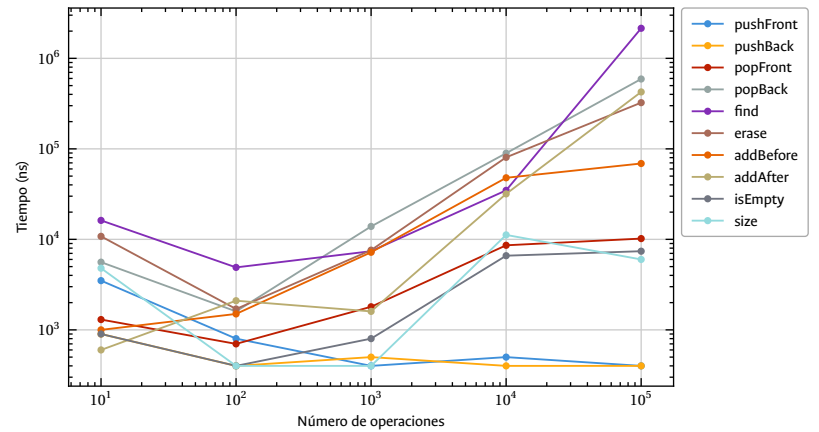
2. Simples (con cola)

Method	10	100	1000	10000	100000
pushFront	9700	500	400	300	400
pushBack	4700	800	300	300	200
popFront	6800	3700	1200	1600	1000
popBack	2600	1500	10700	97200	957500
find	6800	2600	31900	235200	2207500
erase	6300	3900	22000	96100	696600
addBefore	600	1500	7200	69000	361300
addAfter	600	1700	1600	46300	287600
isEmpty	300	200	400	400	1400
size	1500	300	300	400	2300



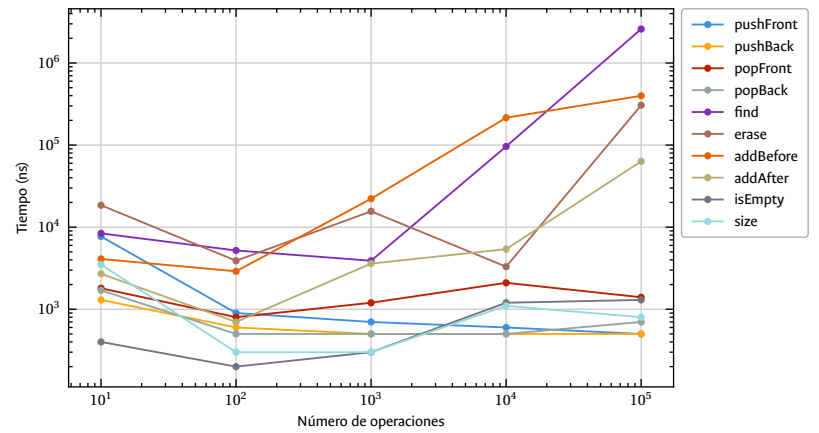
3. Dobles

Method	10	100	1000	10000	100000
pushFront	3500	800	400	500	400
pushBack	900	400	500	400	400
popFront	1300	700	1800	8600	10200
popBack	5600	1600	13900	89500	591500
find	16200	4900	7400	34800	2149700
erase	10800	1700	7600	80600	324000
addBefore	1000	1500	7200	47900	68900
addAfter	600	2100	1600	31800	425600
isEmpty	900	400	800	6600	7400
size	4800	400	400	11200	6000



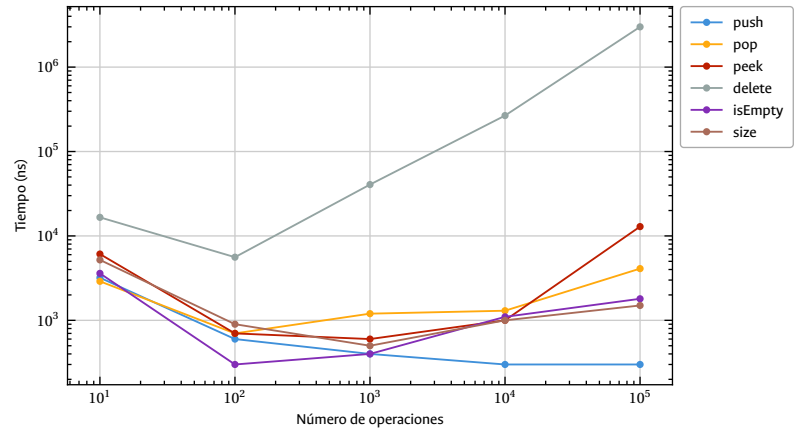
4. Dobles (sin cola)

Method	10	100	1000	10000	100000
pushFront	7700	900	700	600	500
pushBack	1300	600	500	500	500
popFront	1800	800	1200	2100	1400
popBack	1700	500	500	500	700
find	8400	5200	3900	96200	2593200
erase	18500	3900	15600	3300	306000
addBefore	4100	2900	22200	215500	398500
addAfter	2700	700	3600	5400	63300
isEmpty	400	200	300	1200	1300
size	3500	300	300	1100	800



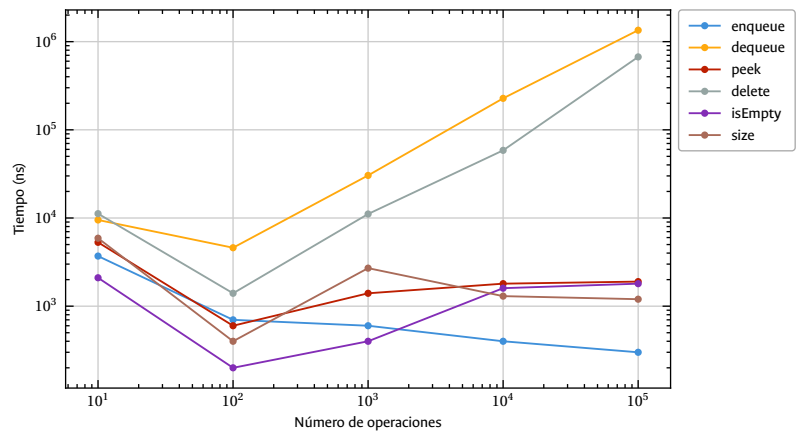
Pilas

Method	10	100	1000	10000	100000
push	3200	600	400	300	300
pop	2900	700	1200	1300	4100
peek	6100	700	600	1000	12900
delete	16600	5600	40600	266600	2997300
isEmpty	3600	300	400	1100	1800
size	5200	900	500	1000	1500



Colas

Method	10	100	1000	10000	100000
enqueue	3700	700	600	400	300
dequeue	9500	4600	30400	227500	1347600
peek	5300	600	1400	1800	1900
delete	11200	1400	11100	58500	671200
isEmpty	2100	200	400	1600	1800
size	5900	400	2700	1300	1200



Conclusiones

Hemos hecho un análisis teórico y empírico sobre el funcionamiento de las listas enlazadas en todas sus formas, las pilas y las colas y hecho supuestos en base a su funcionamiento y a análisis simples.

La experimentación demostró que la complejidad de las estructuras de datos coincidió con lo esperado y demostrado a lo largo de este informe, gracias a la implementación y medición hechos en Java.

Estos resultados son esenciales en el manejo real de estructuras de datos, las cuales se usan en todos los casos de uso posibles dentro del software real. Su comprensión nos permitirá elegir la mejor estructura para cada contexto.